CSE-1003 • MODULE 3 • CLASS 2

Copy or key?

What really happens when you call a function: copies, addresses, the stack, who can see a name, and how long a variable lives.

TODAY'S SYLLABUS TOPICS

Five topics, one question: copy or key?

1



Call by value

The function gets a copy

2



Call by reference

The function gets the key

3



The stack

A pile of plates

4



Scope

Who can see which name

5



Storage classes

How long a variable lives

Recursion comes next class. Today we learn what happens inside a call.

THE MYSTERY FROM LAST CLASS

add1(a) ran. So why is a still 5?

```c
#include <stdio.h>

void add1(int x) {
    x = x + 1;
}

int main(void) {
    int a = 5;
    add1(a);
    printf("a = %d\n", a);
    return 0;
}
```

What does the program print?

a = 5

a = 6

Vote with your hand: left or right?

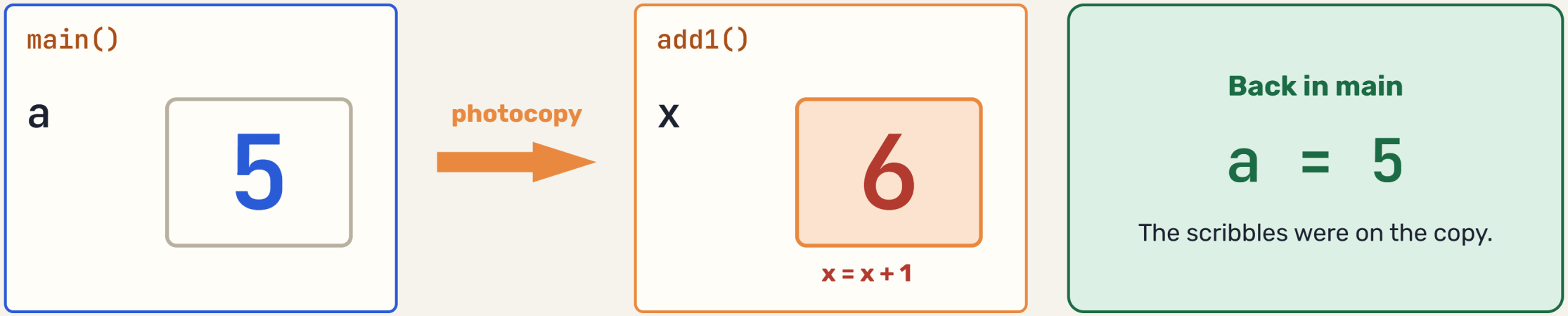
```
a = 5
```

add1 changed something, but not **a**. What did it change?

Today We find out why, and how to make it **6**.

TOPIC 1 OF 5 • CALL BY VALUE

Call by value: the function gets a photocopy



Your original answer sheet

The function's copy

Call by value The function receives a **copy** of the value. Changing the copy never changes the original.

TOPIC 1 OF 5 • CALL BY VALUE • LIKE A DEBUGGER

Call by value: watch the copy

```
1  #include <stdio.h>
2
3  void add1(int x) {
4      x = x + 1;
5  }
6
7  int main(void) {
8      int a = 5;
9      add1(a);
10     printf("a = %d\n", a);
11     return 0;
12 }
```

Memory: the call stack

add1()

x 6 @2000

✗ the box, and its 6, are gone

main()

a 5 @1000

Output

```
a = 5
```

The line being run

A box that just changed

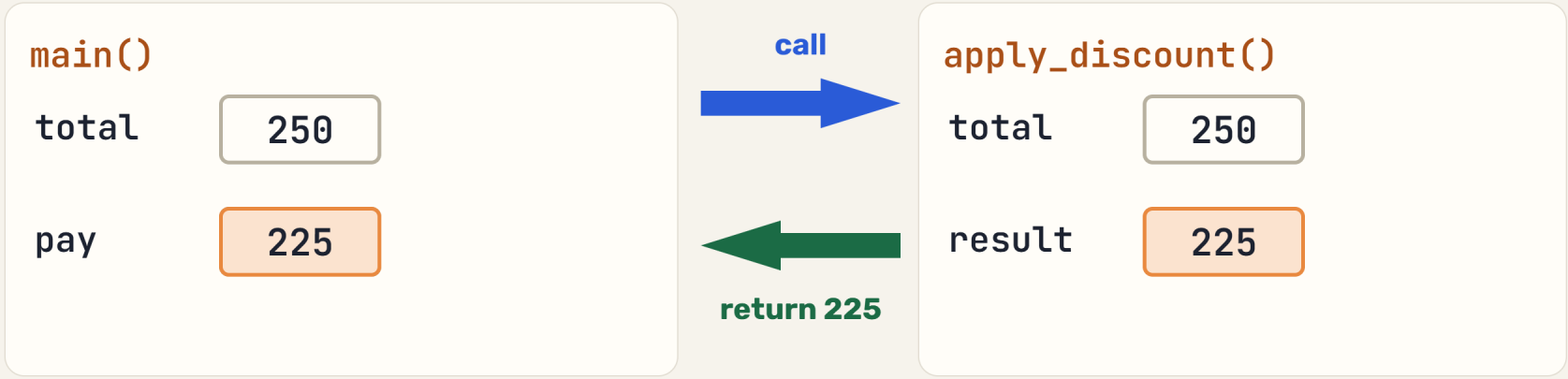
Each box = one function's memory

@2000 is not @1000: x and a are different boxes.

5 printf shows a: still 5.

TOPIC 1 OF 5 • CALL BY VALUE

Call by value: the answer comes back with return




```
in main:
pay =
apply_discount(tota
l);
```

So we use return

The function cannot change **total**. It works on a copy and sends **one** answer back, and main keeps it in **pay**.

TOPIC 1 OF 5 • CALL BY VALUE • PREDICT

What does it print?

1

```
void inc(int n) { n = n + 10; }
int a = 1; inc(a); printf("%d", a);
```

1: inc changed its copy

2

```
int dbl(int n) { return n * 2; }
int a = 4; dbl(a); printf("%d", a);
```

4: the answer was thrown away

3

```
int dbl(int n) { return n * 2; }
int a = 4; a = dbl(a); printf("%d", a);
```

8: the answer is stored back

4

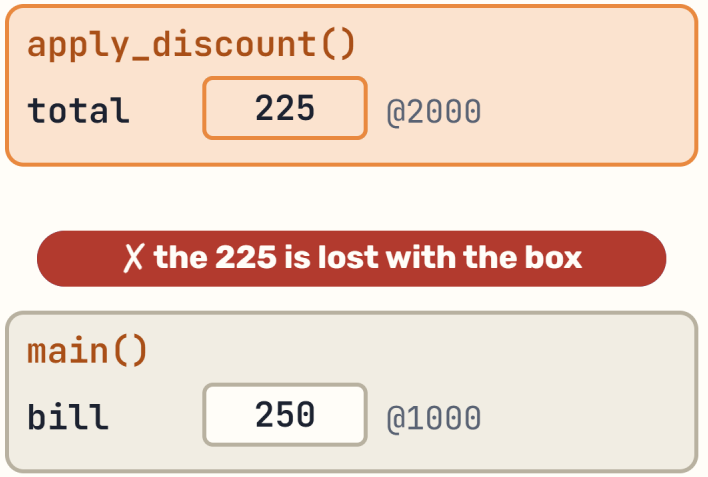
```
int a = 4; int b = dbl(a);
printf("%d %d", a, b);
```

4 8: a is unchanged, b is new

The discount that never arrives

```
1  #include <stdio.h>
2
3  void apply_discount(int total) {
4      if (total > 200)
5          total = total - total / 10;
6  }
7
8  int main(void) {
9      int bill = 250;
10     apply_discount(bill);
11     printf("Pay: Rs %d\n", bill);
12     return 0;
13 }
```

Memory: the call stack



Output

```
Pay: Rs 250
```

✗ should be 225!

The line being run

A box that just changed

Each box = one function's memory

The function did its job, but on a copy.

5 main prints bill: still 250. The customer is overcharged!

TOPIC 2 OF 5 · CALL BY REFERENCE · THE FIX

Give the function the key

```
1  #include <stdio.h>
2
3  void apply_discount(int *total) {
4      if (*total > 200)
5          *total = *total - *total / 10;
6  }
7
8  int main(void) {
9      int bill = 250;
10     apply_discount(&bill);
11     printf("Pay: Rs %d\n", bill);
12     return 0;
13 }
```

Memory: the call stack



Output

```
Pay: Rs 225
```

✓ correct!

- The line being run
- A box that just changed
- Each box = one function's memory

total holds 1000, the address of bill.

6 main prints bill: Rs 225. The customer pays the right amount!

TOPIC 2 OF 5 • CALL BY REFERENCE • RECAP

Same bug, two fixes

The bug: a copy

The function

```
void apply_discount(int total)
```

The call in main

```
apply_discount(bill);
```

The copy changes, bill does not

Rs 250 ✗

Fix 1: return

The function

```
int apply_discount(int total)
```

The call in main

```
bill = apply_discount(bill);
```

One answer comes back

Rs 225 ✓

Fix 2: reference

The function

```
void apply_discount(int *total)
```

The call in main

```
apply_discount(&bill);
```

The function changes bill itself

Rs 225 ✓

The same bug hides in many real programs

- Wallet balance + cashback
- Game score + bonus points
- Stock count after a sale

TOPIC 2 OF 5 • CALL BY REFERENCE

Call by reference: the function gets the key



A photocopy

You get a copy of the value.

`add1(a)`

Edits stay with the copy



The locker key

You get the address of the box.

`add1(&a)`

Edits happen in the original



The locker is the variable a

Locker number 1000 holds the value 5.



The key says 1000

Whoever holds the key can open the locker and change what is inside.

TOPIC 2 OF 5 · CALL BY REFERENCE

Two new symbols: & and *

& — **the address of**

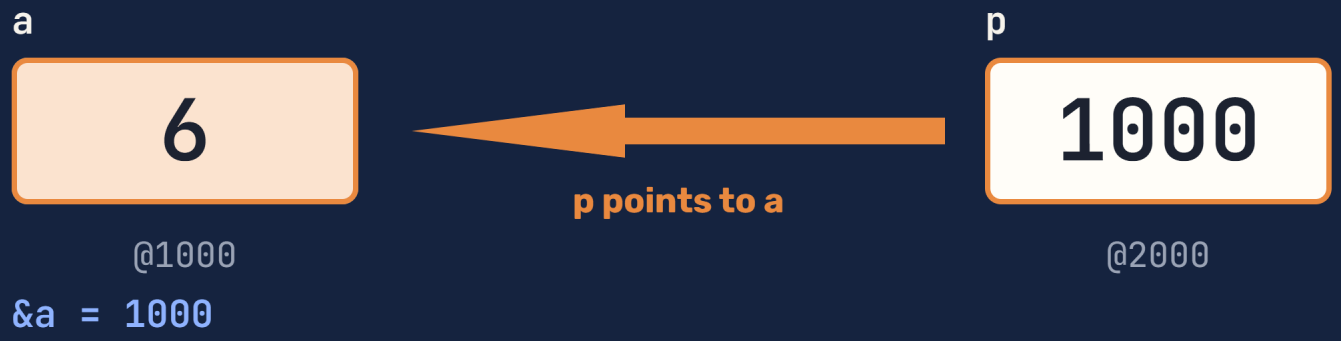
`&a is 1000`

Like a house number.

***** — **go to that address**

`*p is the box at 1000`

Like opening the locker.



`*p = 6;`

Go to address 1000 and store 6 there.
The box a changes!

Call by reference: watch the key at work

```
1  #include <stdio.h>
2
3  void add1(int *x) {
4      *x = *x + 1;
5  }
6
7  int main(void) {
8      int a = 5;
9      add1(&a);
10     printf("a = %d\n", a);
11     return 0;
12 }
```

Memory: the call stack

add1()

x | 1000 | @2000 → a

▲ *x reaches back into main

main()

a | 6 | @1000

Output

```
a = 6
```

The line being run

A box that just changed

Each box = one function's memory

x holds 1000, the address of a. *x means the box at 1000.

5 printf shows a: now 6.

Swap: two keys, two boxes changed

```
1  #include <stdio.h>
2
3  void swap(int *p, int *q) {
4      int t = *p;
5      *p = *q;
6      *q = t;
7  }
8
9  int main(void) {
10     int x = 3, y = 7;
11     swap(&x, &y);
12     printf("x=%d y=%d\n", x, y);
13     return 0;
14 }
```

Memory: the call stack

swap()

p	1000	@2000	→ x
q	1004	@2004	→ y
t	3	@2008	

▼ two addresses go in

main()

x	7	@1000
y	3	@1004

Output

```
x=7 y=3
```

The line being run

A box that just changed

Each box = one function's memory

Last class swap by value failed. Now p and q reach main's real boxes.

7 printf shows the swapped values.

One function, two answers

```
1  #include <stdio.h>
2
3  void make_change(int amt, int *notes,
4                   int *coins) {
5      *notes = amt / 20;
6      *coins = amt % 20;
7  }
8
9  int main(void) {
10     int n, c;
11     make_change(65, &n, &c);
12     printf("%d notes, Rs %d\n", n, c);
13     return 0;
14 }
```

Memory: the call stack

make_change()

amt	65	@2000	
notes	1000	@2004	→ n
coins	1004	@2008	→ c

▼ 1 value and 2 keys go in

main()

n	3	@1000
c	5	@1004

Output

```
3 notes, Rs 5
```

The line being run

A box that just changed

Each box = one function's memory

return can send back only ONE value. Keys let a function fill in many.

6 printf shows both answers, the notes and the coins.

TOPIC 2 OF 5 • CALL BY REFERENCE

You already used call by reference: scanf

```
scanf("%d", &qty);
```



Why the &? scanf must store the number INSIDE qty. It needs the key (address) to qty's box, not a copy of its value.

printf only reads the value, so it takes qty with no &. scanf writes, so it takes &qty.

TOPIC 2 OF 5 • CALL BY REFERENCE

Copy or key? The comparison

	Call by value	Call by reference
The function receives	a copy of the value	the address: a key
Can it change main's variable?	✗ No	✓ Yes
The call looks like	add1(a)	add1(&a)
Inside the function	x	*x
Use it when	you only need an answer back (return)	you must change a variable, or give back several answers

TOPIC 2 OF 5 • CALL BY REFERENCE • PREDICT

What does it print?

1

```
void f(int *p) { *p = 9; }
int a = 1; f(&a); printf("%d", a);
```

9: f reached into a

2

```
void f(int p) { p = 9; }
int a = 1; f(a); printf("%d", a);
```

1: only the copy changed

3

```
int a = 2, b = 5; swap(&a, &b);
printf("%d %d", a, b);
```

5 2: swapped

4

```
scanf("%d", &n); Why the &?
```

scanf needs n's address to store the number there

REAL LIFE • ATTENDANCE SHEET

Which way changes the teacher's sheet?



Send a PDF copy on WhatsApp

The monitor gets a copy

Teacher's sheet

`Absent: 0`

copy

Monitor's copy

`Absent: 1`

Teacher's sheet is still `Absent: 0`

```
mark_absent(absent)
```

PDF copy = call by value



Share the Google Sheet link

The monitor opens the real sheet

Teacher's sheet

`Absent: 1`

link

Monitor

edits the ORIGINAL

Teacher's sheet now says `Absent: 1`

```
mark_absent(&absent)
```

Shared link = call by reference

YOUR TURN · PARSONS PROBLEM

Put the lines in order to build swap

```
A   *p = *q;
B   int t = *p;
C   *q = t;
D   void swap(int *p, int *q) {
E   }
```

swap should exchange the two boxes that p and q point to.

Write the order of letters. Hint: start with the header.

D · B · A · C · E

Save first, then overwrite. If A came before B, x's old value would be lost and both boxes would end up with y's value.

TOPIC 2 OF 5 • CALL BY REFERENCE • MISSING LINE CHALLENGE

One line is missing: make it print a = 6

```c
/* One line is missing. Find it and write it. */
#include <stdio.h>

void add1(int *x) {
    /* ??? missing line ??? */
}

int main(void) {
    int a = 5;
    add1(&a);
    printf("a = %d\n", a);
    return 0;
}
```

The function receives the **address** of a. Write the line that adds 1 to the box it points to.

Correct output

```
a = 6
```

Answer: *x = *x + 1;

Without the stars, x = x + 1 only changes the key (the address), not the box it opens.

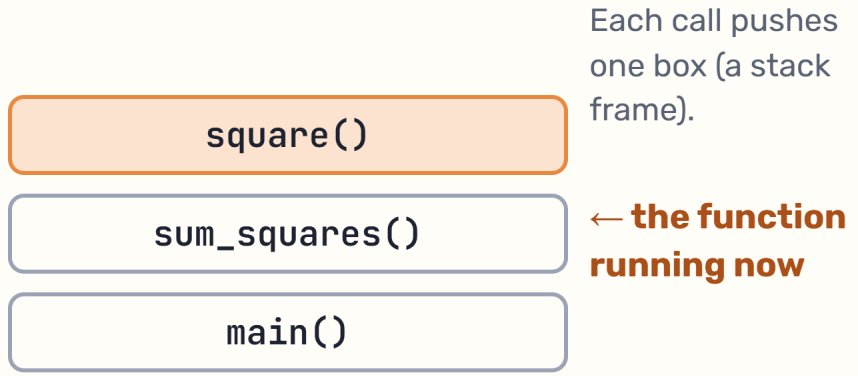
TOPIC 3 OF 5 • THE STACK

The stack: a pile of plates

The canteen plate pile



The call stack



Last in, first out The function called last is on top, so it finishes first. A return pops its box.

The stack grows and shrinks

```
1  #include <stdio.h>
2
3  int square(int n) {
4      return n * n;
5  }
6
7  int sum_squares(int a, int b) {
8      return square(a) + square(b);
9  }
10
11 int main(void) {
12     int r = sum_squares(3, 4);
13     printf("%d\n", r);
14     return 0;
15 }
```

Memory: the stack (grows upward)

Stack depth: 1

`square()`

n	4	@3000

`sum_squares()`

a	3	@2000
b	4	@2004

`main()`

r	25	@1000

Output

```
25
```

The line being run

A box that just changed

Each box = one function's memory

square runs twice in the same place: a box is pushed, popped, pushed again.

9 printf shows 25. Finally main's box goes too.

A debugger session, simulated

```
1  #include <stdio.h>
2
3  int square(int n) {
4      return n * n;
5  }
6
7  int sum_squares(int a, int b) {
8      return square(a) + square(b);
9  }
10
11 int main(void) {
12     int r = sum_squares(3, 4);
13     printf("%d\n", r);
14     return 0;
15 }
```

Variables

in [illegible]

`r = 25`

`b = 4`

Call stack (top = running now)

`main()`

`main()`

`main()`

Controls

Continue | Step Over | Step Into | Step Out

[illegible]

Output

25

9 Step Over: printf runs and the output 25 appears.

TOPIC 3 OF 5 • THE STACK • PREDICT

The stack, in your own words

1	main calls f, f calls g, g calls h. How many boxes are on the stack while h runs?	4: main, f, g and h
2	Which function finishes first: the first one called, or the last one called?	The last one called: it is on top
3	When square returns, which box disappears?	square's box, and its variables with it
4	What lives inside each box on the stack?	That call's own parameters and local variables

TOPIC 4 OF 5 • SCOPE OF VARIABLES

Scope: who can see which name?

The whole campus: global



Campus notice board

Everyone on campus can read it.

One classroom: local

Classroom notice

Only people inside that room can read it.

In C

Global variable: declared outside every function. Every function can use it.

Local variable: declared inside a function (or a block). Only that function can use it.

TOPIC 4 OF 5 · SCOPE OF VARIABLES

Who can see what?

```
1  #include <stdio.h>
2
3  int rate = 10;
4
5  void show(void) {
6      int t = 5;
7      printf("%d %d\n", rate, t);
8  }
9
10 int main(void) {
11     int a = 3;
12     show();
13     printf("%d\n", t);
14     return 0;
15 }
```

	show()	main()
rate global	✓ can see	✓ can see
t local to show	✓ can see	✗ cannot
a local to main	✗ cannot	✓ can see

```
error: 't' undeclared (first use in function main)
```

Line 13: main tries to print t, but t lives only inside show().

TOPIC 4 OF 5 • SCOPE OF VARIABLES • FIND THE BUG

Why is the total still 0?

```
1  #include <stdio.h>
2
3  int total = 0;
4
5  void add_item(int price) {
6      int total = 0;
7      total = total + price;
8  }
9
10 int main(void) {
11     add_item(10);
12     add_item(15);
13     printf("Total: %d\n", total);
14     return 0;
15 }
```

Memory

Global (static area)

total	0	@3000

add_item()

price	15	@2000	
total	15	@2004	**local!**

Output

```
Total: 0
```

The line being run

A box that just changed

Each box = one function's memory

Two different variables, same name: the local one hides the global.

7 The fix: delete the word int on line 6. Then add_item changes the global total.

TOPIC 4 OF 5 • SCOPE OF VARIABLES • PREDICT

What does it print?

1

```
int x = 5; void f(void) { int x = 9; }
main: f(); printf("%d", x);
```

5: f changed its own x

2

Can main use a variable declared inside f?

No: compile error

3

```
for (i = 0; i < 3; i++) { int k = i; }
printf("%d", k);
```

Error: k lives only inside the braces

4

Two functions both declare int i.
Are they the same box?

No: two separate local boxes

TOPIC 5 OF 5 • STORAGE CLASSES

Four storage classes: how a variable lives



`auto`

A sticky note

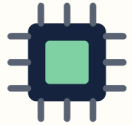
Made when the function starts, thrown away when it ends. The default for local variables.



`static`

A notebook

Kept between calls. It remembers its value for the whole program.



`register`

The fast lane

A request to keep it in the CPU for speed. The compiler may ignore it.



`extern`

Another room

Declared here, but created in another file. It already exists.

TOPIC 5 OF 5 • STORAGE CLASSES • AUTO

The token machine that forgets

```
1  #include <stdio.h>
2
3  int next_token(void) {
4      int t = 0;
5      t++;
6      return t;
7  }
8
9  int main(void) {
10     printf("%d ", next_token());
11     printf("%d ", next_token());
12     printf("%d ", next_token());
13     return 0;
14 }
```

Memory

Static area

next_token()

t 1 @2000

Output

1 1 1

The line being run

A box that just changed

Each box = one function's memory

An auto variable is born at the call and dies at the return.

6 return 1. Three calls, three tokens, all number 1. The machine forgets!

TOPIC 5 OF 5 • STORAGE CLASSES • STATIC

The token machine that remembers

```
1  #include <stdio.h>
2
3  int next_token(void) {
4      static int t = 0;
5      t++;
6      return t;
7  }
8
9  int main(void) {
10     printf("%d ", next_token());
11     printf("%d ", next_token());
12     printf("%d ", next_token());
13     return 0;
14 }
```

Memory

Static area

t 3 @3000

next_token()
no t of its own: it uses the static t

Output

1 2 3

The line being run

A box that just changed

Each box = one function's memory

static keeps one box for the whole program, outside the stack.

7 return 3. The machine remembers: 1 2 3.

TOPIC 5 OF 5 • STORAGE CLASSES

Scope and lifetime at a glance

Storage class	Lives in	Starts as	Can be seen in	Lives until
auto	the stack	garbage	the block	until the block ends
static	the static area	0	the block (or the file)	the whole program
register	a CPU register, if granted	garbage	the block	until the block ends
extern	the static area	0	the whole program	the whole program

Scope: where the name can be used.

Lifetime: how long the box exists.

TOPIC 5 OF 5 • STORAGE CLASSES

Two more: extern and register



extern

It lives in another file

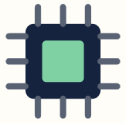
file1.c

```
int visits = 0;
```

file2.c

```
extern int visits;
```

extern says: do not create it here, it already exists elsewhere. Both files then share one box.



register

A request for the fast lane

```
register int i;
```

The compiler may keep i in a CPU register, which is faster. It may also ignore the request.

Rule: you cannot take its address. &i is an error, because a register has no memory address.

TOPIC 5 OF 5 • STORAGE CLASSES • PREDICT

What does it print?

1 `static int t = 0; t++; return t;`
Called 3 times. The three values?

1 2 3: static remembers

2 The same function, but `int t = 0;` (auto).
The three values?

1 1 1: auto forgets

3 Which storage class keeps its value between calls?

static

4 What does a global or static int start as?
And a local auto int?

0, and garbage

PUTTING IT ALL TOGETHER

Canteen v2: every idea from today

`main()` calls **serve()** once for each of 3 customers

local	`total, discount, pay`	Belong to serve(). Every customer gets fresh ones.
call by value	`item_total(qty, price)`	Copies go in. The answer comes back with return.
call by reference	`bill(total, &discount, &pay)`	Two addresses go in. Two answers are written into serve().
static	`next_token()`	A counter that remembers: 1, 2, 3.
global	`items_sold`	One total, shared by every function.

THE PROGRAM, PART 1 OF 3

Announce, then tell the story

3 **global: items_sold lives outside every function**

5–8 **prototypes: the menu board for all four functions**

10–16 **main tells the story: three customers, then the global total**

```
1  #include <stdio.h>
2
3  int items_sold = 0;
4
5  int next_token(void);
6  int item_total(int qty, int price);
7  void bill(int total, int *discount, int *pay);
8  void serve(int chai, int samosa, int maggi);
9
10 int main(void) {
11     serve(2, 1, 1);
12     serve(10, 10, 0);
13     serve(0, 0, 6);
14     printf("Items sold today: %d\n", items_sold);
15     return 0;
16 }
```

THE PROGRAM, PART 2 OF 3

Three helper functions

18–22 **static t: one box that remembers**

24–26 **by value: qty and price are copies, return brings the answer**

28–33 **by reference: discount and pay are addresses, two answers written back**

```
18  int next_token(void) {
19      static int t = 0;
20      t++;
21      return t;
22  }
23
24  int item_total(int qty, int price) {
25      return qty * price;
26  }
27
28  void bill(int total, int *discount, int *pay) {
29      *discount = 0;
30      if (total > 200)
31          *discount = total / 10;
32      *pay = total - *discount;
33  }
```

THE PROGRAM, PART 3 OF 3

serve ties it together

36 **local: total, discount, pay belong to serve**

37–39 **three calls by value, added up**

40 **&discount, &pay: the keys are handed over**

41 **changes the global items_sold**

42–43 **next_token() gives the next static number**

```
35  void serve(int chai, int samosa, int maggi) {
36      int total, discount, pay;
37      total = item_total(chai, 10)
38            + item_total(samosa, 15)
39            + item_total(maggi, 40);
40      bill(total, &discount, &pay);
41      items_sold += chai + samosa + maggi;
42      printf("Token %d: total %d, discount %d, pay %d\n",
43             next_token(), total, discount, pay);
44  }
```

RUN IT YOURSELF

Run the canteen, then check your predictions

`canteen_v2.c`

1. Open the link or scan the QR code
2. This program needs no input
3. Press **Run**
4. Check the last line: **Items sold today: 30**



onlinegdb.com/9kiVY7a9P

PREDICT, THEN RUN TO CHECK

What will the canteen do?

1	What token does the third customer get?	3: the static counter remembers
2	serve(0, 0, 6): total, discount, pay?	240, 24 and 216
3	What is items_sold after all three customers?	30: 4 + 20 + 6
4	Which function writes into discount and pay?	bill, through their addresses

YOUR TURN • TASK 1 OF 3

Smaller and larger



A function can return only one value. Write min_max that gives back BOTH the smaller and the larger of two numbers.

The function recipe

1	**Name**	`min_max`
2	**Inputs**	`int a, int b`
3	**Gives back**	`two answers, through addresses`
4	**Header**	`void min_max(int a, int b, int *small, int *big)`
5	**Body**	`compare a and b, store both with *`
6	**Test**	`min_max(8, 3, &lo, &hi) gives 3 and 8`

Sample runs

```
Smaller = 3
Larger = 8

min_max(5, 5, ...)
Smaller = 5
Larger = 5
```

Check yourself

- ✓ Two answers: two addresses
- ✓ Test equal numbers: 5 and 5
- ✓ Which lines need the *?

Now: Write it in C, then test every sample.

SOLUTION · TASK 1: MIN_MAX()

min_max gives two answers back

```c
#include <stdio.h>

void min_max(int a, int b,
             int *small, int *big) {
    if (a < b) { *small = a; *big = b; }
    else { *small = b; *big = a; }
}

int main(void) {
    int lo, hi;
    min_max(8, 3, &lo, &hi);
    printf("Smaller = %d\n", lo);
    printf("Larger = %d\n", hi);
    return 0;
}
```

Memory

`min_max()`

a	8	@2000	
b	3	@2004	
small	1000	@2008	→ lo
big	1004	@2012	→ hi

`main()`

lo	3	@1000	
hi	8	@1004	

Output

```
Smaller = 3
Larger = 8
```

The line being run

A box that just changed

Each box = one function's memory

Two keys let one function fill in two boxes.

6 printf shows both answers.

YOUR TURN • TASK 2 OF 3

Roll call numbers



Each call should give the next roll number: 1, then 2, then 3. The function must remember its count between calls.

The function recipe

1	**Name**	`next_roll`
2	**Inputs**	`none`
3	**Gives back**	`int: the next roll number`
4	**Header**	`int next_roll(void)`
5	**Body**	`a static counter: add 1, then return it`
6	**Test**	`three calls give 1, 2, 3`

Sample runs

```
Roll 1
Roll 2
Roll 3
```

Check yourself

- ✓ Which keyword makes it remember?
- ✓ Is the 0 set every call, or once?
- ✓ Call it three times from main

Now: Write it in C, then test every sample.

SOLUTION · TASK 2: NEXT_ROLL()

next_roll remembers

```
1  #include <stdio.h>
2
3  int next_roll(void) {
4      static int roll = 0;
5      roll++;
6      return roll;
7  }
8
9  int main(void) {
10     printf("Roll %d\n", next_roll());
11     printf("Roll %d\n", next_roll());
12     printf("Roll %d\n", next_roll());
13     return 0;
14 }
```

Memory

Static area

roll 3 @3000

next_roll()
no roll of its own: it uses the static roll

Output

```
Roll 1
Roll 2
Roll 3
```

The line being run

A box that just changed

Each box = one function's memory

static keeps one box for the whole program.

7 return 3. Output: Roll 3.

YOUR TURN • TASK 3 OF 3

Seconds to hours, minutes and seconds



Turn a number of seconds into hours, minutes and seconds. Three answers come back, so pass three addresses.

The function recipe

1	**Name**	`to_hms`
2	**Inputs**	`int sec`
3	**Gives back**	`three answers: hrs, mins, secs`
4	**Header**	`void to_hms(int sec, int *hrs, int *mins, int *secs)`
5	**Body**	`/ 3600, then % 3600 / 60, then % 60`
6	**Test**	`to_hms(3725, &h, &m, &s) gives 1, 2, 5`

Sample runs

```
1 h 2 m 5 s

to_hms(59, ...)
0 h 0 m 59 s
```

Check yourself

- ✓ % 60 gives the seconds
- ✓ Test 59 and 3600
- ✓ Three answers: three addresses

Now: Write it in C, then test every sample.

SOLUTION · TASK 3: TO_HMS()

to_hms gives three answers back

```
1  #include <stdio.h>
2
3  void to_hms(int sec, int *hrs,
4              int *mins, int *secs) {
5      *hrs = sec / 3600;
6      *mins = sec % 3600 / 60;
7      *secs = sec % 60;
8  }
9
10 int main(void) {
11     int h, m, s;
12     to_hms(3725, &h, &m, &s);
13     printf("%d h %d m %d s\n", h, m, s);
14     return 0;
15 }
```

Memory

to_hms()

sec	3725	@2000	
hrs	1000	@2004	→ h
mins	1004	@2008	→ m
secs	1008	@2012	→ s

main()

h	1	@1000
m	2	@1004
s	5	@1008

Output

```
1 h 2 m 5 s
```

The line being run

A box that just changed

Each box = one function's memory

Three keys, three boxes filled in.

7 printf shows all three.

TODAY IN ONE SLIDE

Copy or key?

1

Call by value

The function gets a copy. Changing it never changes yours. return brings one answer back.

2

Call by reference

Pass &a, use *x. The function reaches your box, and can give back many answers.

3

The stack

One box per call, a pile of plates. The last called finishes first.

4

Scope

Local names are seen only inside their function. A local can hide a global.

5

Storage classes

auto forgets after each call. static remembers.

A copy keeps your data safe. A key lets the function change it.

NEXT CLASS

What if a function could call itself?

Class 3: recursive functions, the stack growing and shrinking, and recursion versus iteration.

Before next class: do one task, and run stack_demo.c in the debugger.